



ble-sensors-mqtt

Installation and usage manual

Version 1.0.0 - English edition

Alessandro De Salvo - EUPL-1.2

1. Purpose

ble-sensors-mqtt 1.0.0 is a multi-sensor gateway for Raspberry Pi. Plugins collect BLE or Tuya Cloud data and emit one common model containing identifier, name, manufacturer, model, protocol, signal strength, timestamp, and all available values. The same snapshot feeds MQTT, Prometheus, and SNMP.

2. Installation

```
sudo apt update
sudo apt install -y bluetooth bluez python3 python3-venv
sudo systemctl enable --now bluetooth
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install '.*[all]'
```

On a Pi Zero, use `pip install .` for SwitchBot only, `.*[sensors]` for all BLE decoders, or `.*[cloud]` for SwitchBot and Tuya. Verify the result with `ble-sensors-mqtt --list-plugins`.

3. Supported plugins

Name	Family	Access
switchbot	SwitchBot	local BLE
xiaomi	Xiaomi/Mijia/HHCC	BLE, optional bind key
govee	Govee	local BLE
inkbird	Inkbird	BLE/connection on compatible models
thermopro	ThermoPro	local BLE
qingping	Qingping/ClearGrass	local BLE
bthome	BTHome, Shelly BLU, ATC/PVVX-BTHome	local BLE
ruuvi	RuuviTag	local BLE
sensornpush	SensorPush	local BLE
airthings	Airthings	active BLE GATT
mopeka	Mopeka	local BLE
tuya-cloud	Tuya/Smart Life	HTTPS cloud

Support varies by model and firmware. Clear-text BLE sensors do not need to be added to the vendor app. The app may coexist with the gateway and can be required for cloud pairing or for obtaining encryption keys.

4. Scan, selection, and aliases

```
.venv/bin/ble-sensors-mqtt --scan --scan-duration 15
.venv/bin/ble-sensors-mqtt --scan --plugin switchbot --plugin bthome
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 \
--device AA:BB:CC:DD:EE:FF \
--device-name 'AA:BB:CC:DD:EE:FF=Server Room'
```

Scanning needs no MQTT broker. `--device ID` accepts either a BLE MAC or a cloud/plugin identifier. Use `--sensor-name 'ID=NAME'` for Tuya or multi-channel readings. Aliases reach every output while `bluetooth_name` keeps the original radio name.

For encrypted Xiaomi/BTHome frames, store the hexadecimal bind key in a protected file and add `[ble_keys."MAC"], plugin="xiaomi" or "bthome", and bindkey_file="/path/file"` to the configuration TOML. Never pass keys on the command line.

5. Runtime and MQTT

Home Assistant MQTT Discovery

Add `--home-assistant-discovery` to publish retained Home Assistant discovery configuration. The default discovery prefix is `homeassistant`; override it with `--home-assistant-discovery-prefix`. Each exported sensor becomes one Home Assistant device, with one sensor entity for each scalar payload value plus RSSI and protocol diagnostics. State remains on the regular gateway MQTT state topic and availability is tied to `<mqtt-prefix>/bridge/status`.

```
.venv/bin/ble-sensors-mqtt --mqtt-host 127.0.0.1 --home-assistant-discovery
```

Discovery configuration is retained. Stale discovery topics are explicitly cleared using retained empty payloads so removed sensors/entities disappear from Home Assistant. Keep the runtime state file enabled in long-running/systemd deployments so cleanup also works across restarts.

```
.venv/bin/ble-sensors-mqtt --mqtt-host 127.0.0.1 --once
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --poll-interval 60 --allow-insecure-mqtt
.venv/bin/ble-sensors-mqtt --mqtt-host mqtt.example.net --mqtt-port 8883 \
  --mqtt-username sensor-publisher \
  --mqtt-password-file /etc/ble-sensors-mqtt/mqtt-password --mqtt-tls
```

Polling defaults to 30 seconds. TLS certificates are always verified. Secret files must be 0640 or more restrictive. Every payload contains manufacturer, model, protocol, and all decoder information under `data`.

6. Tuya Cloud

Run `--cloud-help`, create a Smart Home project in Tuya IoT, and link the Smart Life/Tuya account. Keep Access ID and Access Secret in separate protected files.

```
[cloud.tuya]
enabled = true
region = "eu"
api_key_file = "/etc/ble-sensors-mqtt/tuya-api-key"
api_secret_file = "/etc/ble-sensors-mqtt/tuya-api-secret"
api_device_id = "REFERENCE_DEVICE_ID"
device_ids = ["DEVICE_ID_1"]
```

The TOML file must be 0640 or more restrictive. Test it with:

```
.venv/bin/ble-sensors-mqtt --scan --cloud-config /etc/ble-sensors-mqtt/cloud.toml
```

A Tuya failure does not stop BLE collection. Review cloud privacy, data residency, and service dependency. Tuya devices running BTHome firmware may work locally.

7. Prometheus

```
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --prometheus
curl http://127.0.0.1:9105/metrics
```

Each sensor series has address, name, manufacturer, model, and protocol labels. Dedicated metrics cover temperature, humidity, battery, and RSSI. Other numbers and booleans use `ble_sensors_sensor_value`; strings use `ble_sensors_sensor_info`.

```
--prometheus --prometheus-host 0.0.0.0 --prometheus-port 9200 --allow-external-prometheus
```

The default is `127.0.0.1:9105`. `0.0.0.0` or `:` permits external access if the firewall allows it. The endpoint has no authentication or TLS; use a VPN, reverse proxy, or network ACL.

8. SNMP

```
.venv/bin/ble-sensors-mqtt --mqtt-host 192.0.2.10 --snmp \
  --snmp-community-file /etc/ble-sensors-mqtt/snmp-community
```

The default is `127.0.0.1:1161`. Use `--snmp-host 0.0.0.0 --allow-external-snmp/::` and `--snmp-port` for external access. Table `.10` includes manufacturer, model, and protocol; `.20` exposes every scalar as key, value, type, and unit. SNMPv2c is not encrypted: restrict UDP access or use a VPN/SNMPv3 proxy. The bundled MIB is `BLE-SENSORS-MQTT-MIB.txt`.

9. systemd

For a complete installation starting from a GitHub clone (host prerequisites, Bluetooth check, venv, and systemd):

```
git clone https://github.com/desalvo/ble-sensors-mqtt.git
cd ble-sensors-mqtt
sudo scripts/install-from-github.sh --mqtt-host mqtt.example.net --prometheus
```

Use the systemd installer directly when the clone and prerequisites already exist:

```
# Interactive; CLI values are the defaults shown by the prompts
sudo scripts/install-systemd.sh --mqtt-host mqtt.example.net --prometheus

# Fully unattended provisioning
sudo scripts/install-systemd.sh --non-interactive \
  --mqtt-host mqtt.example.net --mqtt-tls \
  --mqtt-username ble-sensors-publisher --mqtt-password-file ./mqtt-password \
  --home-assistant-discovery --prometheus
```

The installer creates the service user, isolated venv, root-owned configuration and `/etc/ble-sensors-mqtt/service-args.json`; repeatable arguments and values containing spaces are preserved without shell parsing. In interactive mode, CLI values remain the defaults shown by the wizard and already supplied repeatable options are retained. Use `--non-interactive` to install only from CLI/default values. See `docs/SYSTEMD.en.md`.

10. Docker and Kubernetes

```
# Multiarch build + Docker Hub push of desalvo/ble-sensors-mqtt:<VERSION>
docker login
scripts/build-docker.sh

# Continuous Docker service
cd docker
cp .env.example .env
cp arguments.example arguments
docker compose up -d

# Continuous Kubernetes service
kubectl label node NODE_NAME ble-sensors-mqtt/bluetooth=true
kubectl apply -k kubernetes/
```

Before Docker/Kubernetes, validate the host with `sudo scripts/check-bluetooth-host.sh --strict`. Both mount `/run/dbus` read-only and use host BlueZ over system D-Bus; privileged mode is not required in the supported model. Kubernetes also includes `kubernetes/bluetooth-test-pod.yaml` to validate `--scan` on the selected node.

The image supports `linux/amd64` and `linux/arm64`. Docker/Kubernetes mount the host system D-Bus for BlueZ, persist runtime state, send MQTT outbound, expose Prometheus on TCP/9105 and SNMP on UDP/1161. Optional Kubernetes LoadBalancer manifests expose Prometheus and SNMP externally when explicitly applied. `scripts/build-docker.sh` pushes `desalvo/ble-sensors-mqtt:<VERSION>` by default; CI can publish the versioned image on `vX.Y.Z` tags and `latest` on `main`. See `docs/DOCKER.en.md` and `docs/KUBERNETES.en.md`; external monitoring endpoints must be restricted to trusted networks.

11. Extensions

A third-party local package implements `SensorPlugin.decode()`, returns `SensorReading` objects, and registers itself in the `ble_sensors_mqtt.sensor_plugins` entry-point group. A third-party cloud provider implements `CloudPlugin.poll()` and registers a factory/class in `ble_sensors_mqtt.cloud_plugins`; its constructor receives `(config, read_secret)` and its default TOML section is `[cloud.<entry-point-name-without--cloud>]`. Every reading must contain manufacturer, model, and protocol. Plugin exceptions are isolated.

12. Security and troubleshooting

- do not run as root;
- protect MQTT, Tuya, bind-key, and SNMP files;
- use minimal MQTT ACLs and TLS;

- do not expose Prometheus/SNMP without network controls;
- update and audit dependencies with `pip-audit`;
- test actual sensor models before production.

```
bluetoothctl show
.venv/bin/ble-sensors-mqtt --list-plugins
.venv/bin/ble-sensors-mqtt --scan --scan-duration 20 --log-level DEBUG
journalctl -u ble-sensors-mqtt --since today
```

The AgID review is a technical self-assessment, not a certification.

13. Build and rollback

`scripts/build-package.sh` generates a UTC build ID in YYYYMMDD-HHMM format, ZIP, TAR.GZ, and SHA-256 checksums. Keep the previous package for rollback; do not modify site-packages in place.

14. Production health and release gates

Health and readiness

When Prometheus is enabled, the same HTTP server provides:

- `/metrics` for Prometheus;
- `/healthz` for process liveness;
- `/readyz` for readiness. It returns HTTP 503 until one polling cycle succeeds and again if successful polling becomes stale.

Non-loopback Prometheus/health exposure requires `--allow-external-prometheus`; non-loopback SNMPv2c requires `--allow-external-snmp`. Remote or credentialed plaintext MQTT requires `--allow-insecure-mqtt`. These overrides are deliberate risk acknowledgements, not recommended production defaults.

`--plugin-timeout` limits one decoder/cloud call (default 15 seconds). `--stale-cycles` controls when retained MQTT state for a missing sensor is cleared (default 3 consecutive cycles). `--state-file` persists this cleanup state across restarts; the supplied systemd unit uses `/var/lib/ble-sensors-mqtt/state.json`.

Release verification

Install `. [all, dev, release]` and run:

```
scripts/release-check.sh
```

The gate requires lint, tests/coverage, Bandit, dependency audit, EN/IT PDF generation, wheel and sdist build, `twine check`, CycloneDX SBOM generation, and release package verification. CI repeats runtime tests on Python 3.11, 3.12 and 3.13.

For GitHub publication, push the source commit first and then push a matching `vX.Y.Z` tag. The tag workflow runs all gates, builds deterministic release assets, creates consolidated SHA-256 checksums, and publishes the GitHub Release automatically only after the test and build jobs succeed. The release job checks that the tag version matches `VERSION` and `pyproject.toml`.